

Manual Técnico de Implementación:

Dragon Nutrex

Audiencia: Desarrolladores de Software / Ingeniería de Sistemas

Arquitectura: .NET 8 + Redis Cloud + MVC Pattern

1. Perspectiva del Desarrollador

El proyecto Dragon Nutrex ha sido diseñado bajo principios de alta cohesión y bajo acoplamiento. Como desarrolladores, hemos implementado una arquitectura donde la persistencia NoSQL no es solo un almacenamiento, sino un motor de alta velocidad que soporta la lógica nutricional en tiempo real. La elección de Redis responde a la necesidad de latencias ultra bajas, mientras que el patrón Repository asegura que el código sea testeable y mantenible a largo plazo.

2. Prerrequisitos del Entorno

Para garantizar una implementación determinista, el host debe cumplir con:

- .NET 8.0 SDK (LTS): Framework de ejecución principal.
- Redis Engine 7.2+: Soporte para estructuras HASH y SET.
- Python 3.10+: Para scripts de automatización de datos (Seeding).
- Docker Desktop: Para orquestación de servicios locales.
- Redis Insight: Herramienta de auditoría visual de memoria.

3. Guía de Implementación Paso a Paso (A to Z)

Fase 1: Aprovisionamiento de Infraestructura

Clone el repositorio oficial y levante la instancia de base de datos:

```
git clone [https://github.com/BeatrixxK/Codigo_Entregable_1.git]
```

```
docker run -d --name dragon-redis -p 6379:6379 redis/redis-stack:latest
```

Fase 2: Configuración de la Capa de Datos

Edite el archivo appsettings.json para establecer el handshake con Redis. El uso del patrón Singleton en el ConnectionMultiplexer es obligatorio para optimizar los sockets del servidor.

```
"ConnectionStrings": {  
  "RedisConnection": "localhost:6379,password=tu_clave,abortConnect=false"  
}
```

Fase 3: Inicialización de Datos (The Seeding Script)

Ejecute el script de Python para inyectar el catálogo maestro de productos en Redis. Esto mapea objetos directamente a la RAM mediante comandos HSET.

```
pip install redis  
python seed_data.py
```

Fase 4: Compilación y Despliegue

Restaure paquetes NuGet y ejecute el servidor Kestrel:

```
dotnet restore  
dotnet run --project DragonNutrex.Web
```

4. Conclusión Técnica

La implementación exitosa de esta "receta" tecnológica garantiza un sistema escalable. La separación de responsabilidades entre el Controlador y el Repositorio de



Redis permite que Dragon Nutrex no solo sea una aplicación funcional, sino una plataforma robusta lista para producción. Se recomienda monitorear el uso de memoria en Redis Insight para optimizar los TTL de las sesiones de usuario.